

AnyCloud平台AIC8800D40L SDIO WIFI蓝牙一体模组适配指导手册

前言

AIC8800D40L是AIC出的一款支持SDIO接口的WiFi芯片，支持2.4GHz/5GHz Wi-Fi6无线芯片，同时也支持蓝牙BLE 5.4。

现基于AnyCloud39AV100平台对AIC8800D40L适配进行简要说明，其他平台供参考。

1. 移植WiFi驱动

当前使用的WiFi驱动版本为：`aic8800d_linux_sdk_V3.0_2023_1219_3cf85031.7z`。

解压WiFi驱动包：

```
7z x aic8800d_linux_sdk_V3.0_2023_1219_3cf85031.7z -o.
```

Linux环境下可以直接使用7z命令解压，注意默认中的`.`不可忽略。

AIC8800D40L使用SDIO接口时，对应驱动目录为**SDIO/driver_fw/driver/aic8800**。

1.1 适配Makefile文件

1.1.1 调整顶层Makefile

```
CONFIG_AIC8800_BTLPMP_SUPPORT := n
CONFIG_AIC8800_WLAN_SUPPORT := m
CONFIG_AIC_WLAN_SUPPORT := m
.....
```

```

# Platform support list
CONFIG_PLATFORM_ROCKCHIP = n
CONFIG_PLATFORM_ROCKCHIP2 = n
CONFIG_PLATFORM_ALLWINNER = n
CONFIG_PLATFORM_AMLOGIC = n
CONFIG_PLATFORM_UBUNTU ?= n
CONFIG_PLATFORM_ANYKA ?= y
.....

ifeq ($(CONFIG_PLATFORM_ANYKA), y)
KDIR ?= /home/xiaozijie/project/Anycloud39AV100/V1.08/os/kbd
ARCH := arm
KVER := $(shell uname -r)
PWD := $(shell pwd)
MODDESTDIR := $(PWD)/_install
CROSS_COMPILE ?= arm-anycloud-linux-uclibcgnueabi-
ccflags-y += -DCONFIG_PLATFORM_ANYKA
endif

all: modules
modules:
    make -C $(KDIR) M=$(PWD) ARCH=$(ARCH) CROSS_COMPILE=$(CROSS_COMPILE)
modules

install:
    mkdir -p $(MODDESTDIR)
    install -p -m 644 aic8800_bsp/aic8800_bsp.ko $(MODDESTDIR)/
    install -p -m 644 aic8800_fdrv/aic8800_fdrv.ko $(MODDESTDIR)/
ifeq ($(CONFIG_AIC8800_BTLPM_SUPPORT), m)
    install -p -m 644 aic8800_btlpm/aic8800_btlpm.ko $(MODDESTDIR)/
endif
#    /sbin/depmod -a ${KVER}

uninstall:
    rm -rfv $(MODDESTDIR)/aic8800_bsp.ko
    rm -rfv $(MODDESTDIR)/aic8800_fdrv.ko
ifeq ($(CONFIG_AIC8800_BTLPM_SUPPORT), m)
    rm -rfv $(MODDESTDIR)/aic8800_btlpm.ko
endif
#    /sbin/depmod -a ${KVER}

```

添加Anycloud平台配置,以及屏蔽无用编译命令。

1.1.2 调整aic8800_fdrv目录下Makefile

```

# Platform support list
CONFIG_PLATFORM_ROCKCHIP ?= n
CONFIG_PLATFORM_ROCKCHIP2 ?= n
CONFIG_PLATFORM_ALLWINNER ?= n
CONFIG_PLATFORM_INGENIC_T20 ?= n
CONFIG_PLATFORM_AMLOGIC ?= n
CONFIG_PLATFORM_UBUNTU ?= n
CONFIG_PLATFORM_ANYKA ?= y

.....

ifeq ($(CONFIG_PLATFORM_ANYKA), y)
KDIR ?= /home/xiaozijie/project/Anycloud39AV100/V1.08/os/kbd
ARCH ?= arm
KVER ?= $(shell uname -r)
PWD ?= $(shell pwd)
MODESTDIR ?= $(PWD)/_install
CROSS_COMPILE ?= arm-anycloud-linux-uclibcgnueabi-
ccflags-y += -DCONFIG_PLATFORM_ANYKA
endif

all: modules
modules:
    make -C $(KDIR) M=$(PWD) ARCH=$(ARCH) CROSS_COMPILE=$(CROSS_COMPILE)
modules

```

添加Anycloud平台配置。

1.2.3 调整aic8800_bsp目录下Makefile

```

# Platform support list
CONFIG_PLATFORM_ROCKCHIP ?= n
CONFIG_PLATFORM_ROCKCHIP2 ?= n
CONFIG_PLATFORM_ALLWINNER ?=n
CONFIG_PLATFORM_INGENIC_T20 ?= n
CONFIG_PLATFORM_AMLOGIC ?= n
CONFIG_PLATFORM_UBUNTU ?= n
CONFIG_PLATFORM_ANYKA ?= y
.....

ifeq ($(CONFIG_PLATFORM_ANYKA), y)
KDIR ?= /home/xiaozijie/project/Anycloud39AV100/V1.08/os/kbd
ARCH ?= arm
KVER ?= $(shell uname -r)
PWD ?= $(shell pwd)
MODDESTDIR ?= $(PWD)/_install
CROSS_COMPILE ?= arm-anycloud-linux-uclicgnueabi-
ccflags-y += -DCONFIG_PLATFORM_ANYKA
endif

all: modules
modules:
    make -C $(KDIR) M=$(PWD) ARCH=$(ARCH) CROSS_COMPILE=$(CROSS_COMPILE)
modules

```

添加Anycloud平台配置。

1.2 适配驱动源码

1.2.1 调整aic8800_fdrv目录下驱动

1.2.1.1 调整aic8800_bsp/aic_bsp_main.c

```
#ifdef CONFIG_PLATFORM_ANYKA
int aicwf_dbg_level = LOGERROR|LOGINFO;
#else
int aicwf_dbg_level = LOGERROR|LOGINFO|LOGDEBUG|LOGTRACE;
#endif

module_param(aicwf_dbg_level, int, 0660);
```

调整默认打印调试等级。

1.2.2 调整aic8800_bsp目录下驱动

1.2.2.1 调整aic8800_bsp/aic_bsp_main.c

```
#ifdef CONFIG_PLATFORM_ANYKA
int aicwf_dbg_level_bsp = LOGERROR|LOGINFO;
#else
int aicwf_dbg_level_bsp = LOGERROR|LOGINFO|LOGDEBUG|LOGTRACE;
#endif
```

调整默认打印调试等级。

1.2.2.2 调整aic8800_bsp/aicsdio.c

```

#if 0
    if (host->ios.timing == MMC_TIMING_UHS_DDR50) {
        val = 0x21;//0x1D;//0x5;
    } else {
        val = 0x01;//0x19;//0x1;
    }
    val |= SDIOCLK_FREE_RUNNING_BIT;
    sdio_f0_writeb(sdiodev->func, val, 0xF0, &ret);
    if (ret) {
        sdio_err("set iopad ctrl fail %d\n", ret);
        sdio_release_host(sdiodev->func);
        return ret;
    }
    sdio_f0_writeb(sdiodev->func, 0x0, 0xF8, &ret);
    if (ret) {
        sdio_err("set iopad delay2 fail %d\n", ret);
        sdio_release_host(sdiodev->func);
        return ret;
    }
    sdio_f0_writeb(sdiodev->func, 0x40, 0xF1, &ret);
    if (ret) {
        sdio_err("set iopad delay1 fail %d\n", ret);
        sdio_release_host(sdiodev->func);
        return ret;
    }
    msleep(1);
#endif
#if 1//SDIO CLOCK SETTING
    if ((feature.sdio_clock > 0) && (host->ios.timing !=
MMC_TIMING_UHS_DDR50)) {
        host->ios.clock = feature.sdio_clock;
        host->ops->set_ios(host, &host->ios);
        sdio_dbg("Set SDIO Clock %d MHz\n", host->ios.clock/1000000);
    }
#endif
#else
#ifdef CONFIG_PLATFORM_ANYKA
    if ((feature.sdio_clock > 0) && (host->ios.timing != MMC_TIMING_UHS_DDR50)) {
        host->ios.clock = feature.sdio_clock;
        host->ops->set_ios(host, &host->ios);
        sdio_dbg("%s Set SDIO Clock %d MHz\n", __func__, host->ios.clock/1000000);
    }
#endif
#endif
#endif

```

1.2.2.3 调整aic8800_bsp/aic_bsp_driver.h

```
#define AIC_IRQ_WAKE_FLAG          0          // 0: rising edge, 1: falling edge
#define FEATURE_SDIO_CLOCK         500000000 // 0: default, other: target clock
rate
#ifdef CONFIG_PLATFORM_ANYKA
#define FEATURE_SDIO_CLOCK_V3      250000000 //500000000
#else
#define FEATURE_SDIO_CLOCK_V3      1500000000 // 0: default, other: target clock
rate
#endif
#define FEATURE_SDIO_PHASE         2          // 0: default, 2: 180°
```

调整默认SDIO时钟为25Mhz。

2. WiFi使用

2.1 内核配置

WiFi芯片需要加载固件，此时需要使用内核firmware_class.ko，在内核中配置如下选项:

```
Device Drivers --->
  Generic Driver Options --->
    <M> Userspace firmware loading support
    [*] Include in-kernel firmware blobs in kernel binary (NEW)
    () External firmware blobs to build into the kernel binary (NEW)
```

重新编译内核: `make O=../kld uImage modules`得到firmware_class.ko。

2.2 WiFi驱动加载

将编译好驱动放置于设备端，执行以下命令:

```
modprobe cfg80211
modprobe mmc_core
insmod /usr/modules/ak_mci.ko
sleep 2
insmod /lib/modules/4.4.282/kernel/drivers/base/firmware_class.ko
insmod /usr/modules/aic8800_bsp.ko aic_fw_path=/usr/lib/firmware
insmod /usr/modules/aic8800_fdrv.ko
```

其中/usr/lib/firmware存放WiFi驱动包中SDIO/driver_fw/fw/aic8800D80目录下的文件：

```
aic_userconfig_8800d80.txt
fmacfw_8800d80_u02.bin
fw_adid_8800d80_u02.bin
fw_patch_8800d80_u02.bin
fw_patch_table_8800d80_u02.bin
lmacfw_rf_8800d80_u02.bin
```

2.3 WiFi联网

将/etc/config/wpa_supplicant.conf文件的SSID、密码修改为无线路由器SSID及密码，执行以下命令：

```
wpa_supplicant -Dnl80211 -iwlan0 -c /etc/config/wpa_supplicant.conf -B
route del default
udhcpc -t 30 -T 1 -i wlan0 -b
```

3. 蓝牙使用

3.1 打开Linux内核蓝牙子系统

进入到内核目录，可以通过`make O=../kdb menuconfig`方式进行配置:

```
[*] Networking support --->
  <M> Bluetooth subsystem support --->
    [*] Bluetooth Classic (BR/EDR) features
    [*] Bluetooth Low Energy (LE) features
    Bluetooth device drivers --->
      <M> HCI UART driver
        [*] UART (H4) protocol support
        [*] Three-wire UART (H5) protocol support
```

如果需要调试蓝牙，可以加入蓝牙的debugfs:

```
[*] Export Bluetooth internals in debugfs
```

配置完成后，执行`make O=../kdb modules`命令，内核会生成`crc16.ko`、`bluetooth.ko`、`hci_uart.ko`。

3.2 蓝牙驱动加载

将编译好的驱动打包到文件系统内，依次加载:

```
insmod /lib/modules/4.4.282/kernel/lib/crc16.ko
insmod /lib/modules/4.4.282/kernel/net/bluetooth/bluetooth.ko
insmod /lib/modules/4.4.282/kernel/drivers/bluetooth/hci_uart.ko
```

3.3 初始化蓝牙设备

初始化蓝牙设备需要使用hciconfig等工具，其来源于bluez工具包,本文档不具体讲述bluez编译过程.

将编译好的bluez工具包命令打包到文件系统内，同时需要设定环境变量让系统识别到hciconfig等工具，环境变量设置参考脚本：

```
#!/bin/sh

export LD_LIBRARY_PATH="/usr/local/lib:$LD_LIBRARY_PATH"
export PATH="/usr/local/bin:/usr/local/libexec/bluetooth:$PATH"
```

注意环境变量脚本路径，使用点命令让其生效：`./export.sh`

3.3.1 hciattach初始化蓝牙

使用hciattach命令初始化蓝牙设备：

```
hciattach -s 1500000 /dev/ttySAK2 any 1500000 flow nosleep &
```

3.3.2 查看蓝牙设备节点

使用hciconfig命令查看蓝牙hci0节点：

```
[root@anyka ]$ hciconfig -a
hci0:  Type: BR/EDR  Bus: USB
      BD Address: 38:73:EA:E6:58:1B  ACL MTU: 1021:9  SCO MTU: 255:4
      DOWN
      RX bytes:616 acl:0 sco:0 events:34 errors:0
      TX bytes:375 acl:0 sco:0 commands:34 errors:0
      Features: 0xbf 0x2e 0x4d 0xfe 0xd8 0x3f 0x7b 0x87
      Packet type: DM1 DM3 DM5 DH1 DH3 DH5 HV1 HV3
      Link policy: RSWITCH SNIFF
      Link mode: SLAVE ACCEPT
```

3.4 蓝牙连接

为了能使bluez工具正常使用，需要将部分配置文件拷贝到对应目录。

- 1.将附件bluez目录的usr/local/etc/dbus-1目录拷贝到设备的/etc目录和设备的/usr/local目录，由于anyka平台/etc和/usr/local目录是只读的，需要在制作rootfs时加入；
- 2.将附件bluez目录的usr/local/share/dbus-1拷贝到/usr/local/share目录，同样在nor flash要在制作rootfs时加入，nand flash可以读写则直接拷贝；
- 3.新增dbus用户，用于dbus运行，默认是messagebus，编译时设定为dbus了。

```
echo dbus:x:500:500::/home/dbus:/bin/sh >> /etc/passwd
```

- 4.系统启动时，在/var目录创建run/dbus目录

```
mkdir -p /var/run/dbus
```

5.运行dbus和bluetoothd服务

运行前必须要执行**hciconfig hci0 up**，让蓝牙节点起来。

```
hciconfig hci0 up  
dbus-daemon --system --print-pid --print-address  
bluetoothd -n -d &
```

6.执行bluetoothctl

```
bluetoothctl
```

7.在bluetoothctl窗口中依次输入如下命令，进行蓝牙广播

```
menu adv
name on
back
advertise on
menu gatt
register-service 11223344-1111-2222-3333-112233445566
#register-serviceyes
yes
register-characteristic 0x1234 read,write
#register-characteristic1
1
register-application
```

8.执行以上命令后，在安卓手机中使用“BLE调试宝” app，苹果手机使用“蓝牙调试助手” app，可以搜索到名为**Bluez 5.60**的蓝牙设备。

连接后在蓝牙服务界面，选择**11223344-1111-2222-3333-112233445566**服务，点击发送箭头，即可进行字符或16进制发送。

发送后，设备端可以收到对应的字符则蓝牙通信正常。